Code Explanation

Transaction Data Transformation and Analysis

This code is designed to transform and analyze transaction data using PySpark. It provides a set of functions to clean and standardize transaction data, enrich it with customer and store information, and calculate various metrics at the customer, category, and regional levels.

The code consists of several functions that work together to provide a comprehensive analysis of transaction data. It starts by cleaning and standardizing the raw transaction data, then enriches it with additional information, and finally calculates various metrics to gain insights into customer behavior and transaction patterns.

Overview of the Code

The code is organized into several functions, each with a specific responsibility. The main functions are: clean_transaction_data, enrich_transaction_data, calculate_customer_metrics, calculate_category_metrics, calculate_regional_metrics, and identify_top_customers. These functions work together to transform and analyze the transaction data.

Step 1: Cleaning Transaction Data
The clean_transaction_data function is responsible for cleaning and standardizing the raw transaction data. It converts the is_online column to a boolean value, handles missing values in the category and payment_method columns, filters out records with null amounts, and standardizes category names.

```
cleaned_df = df.withColumn(
    "is_online",
    when(col("is_online").isin(["true", "True", "TRUE", "1", "y", "Y
", "yes", "Yes"]), True)
    .otherwise(False)
)
cleaned_df = cleaned_df.na.fill({
    "category": "Uncategorized",
    "payment_method": "Unknown"
})
cleaned_df = cleaned_df.filter(col("amount").isNotNull())
cleaned_df = cleaned_df.withColumn(
    "category",
    when(col("category") == "Groceries", "Grocery")
    .when(col("category") == "Food & Dining", "Food")
    .otherwise(col("category"))
)
```

Step 2: Enriching Transaction Data
The enrich_transaction_data function enriches the cleaned transaction data with customer and store information. It joins the transaction data with customer and store reference data, and adds date dimensions such as transaction year, month, day, and day of week.

```
enriched_df = transactions_df.join(
    customer_df.select("customer_id", "customer_segment", "loyalty_t
ier", "age_group"),
    on="customer_id",
```

```
    how="left"
)
enriched_df = enriched_df.join(
    store_df.select("store_id", "store_type", "region"),
    on="store_id",
    how="left"
)
enriched_df = enriched_df.withColumn("transaction_year", year(col("t
ransaction_date")))
    .withColumn("transaction_month", month(col("transaction_date")))

    .withColumn("transaction_day", dayofmonth(col("transaction_date"
)))
    .withColumn("transaction_dow", dayofweek(col("transaction_date")
))
```

Step 3: Calculating Customer Metrics

The calculate_customer_metrics function calculates customer-level metrics from the enriched transaction data. It groups the data by customer ID and calculates metrics such as total transactions, total spend, average transaction value, online spend, and instore spend.

```
customer_metrics = df.groupBy("customer_id")
    .agg(
        count("transaction_id").alias("total_transactions"),
        spark_sum("amount").alias("total_spend"),
        avg("amount").alias("avg_transaction_value"),
        spark_sum(when(col("is_online"), col("amount")).otherwise(0)
).alias("online_spend"),
        spark_sum(when(~col("is_online"), col("amount")).otherwise(0
)).alias("instore_spend")
    )
customer_metrics = customer_metrics.withColumn(
    "online_spend_ratio",
    col("online_spend") / col("total_spend")
)
```

Step 4: Calculating Category Metrics

The calculate_category_metrics function calculates category-level metrics from the enriched transaction data. It groups the data by category and calculates metrics such as transaction count, total amount, and average transaction amount.

```
category_metrics = df.groupBy("category")
    .agg(
        count("transaction_id").alias("transaction_count"),
        spark_sum("amount").alias("total_amount"),
        avg("amount").alias("avg_transaction_amount")
    )
    .orderBy(col("total_amount").desc())
```

Step 5: Calculating Regional Metrics

The calculate_regional_metrics function calculates region-level metrics from the enriched transaction

data. It groups the data by region and calculates metrics such as transaction count, total amount, average transaction amount, and customer count.

```
regional_metrics = df.filter(col("region").isNotNull())
    .groupBy("region")
    .agg(
        count("transaction_id").alias("transaction_count"),
        spark_sum("amount").alias("total_amount"),
        avg("amount").alias("avg_transaction_amount"),
        count(col("customer_id")).alias("customer_count")
    )
    .orderBy(col("total_amount").desc())
```

Step 6: Identifying Top Customers

The identify_top_customers function identifies the top N customers by total spend. It uses a window function to rank customers by total spend and returns the top N customers.

```
window_spec = Window.orderBy(col("total_spend").desc())
top_customers = df.withColumn("rank", dense_rank().over(window_spec)
)
    .filter(col("rank") <= n)
    .drop("rank")
```